

第六课预习报告

boolean.c

- 作用: 演示 `_Bool` 类型变量的使用来控制循环读取用户输入
- 功能: 读取用户输入的一系列整数并累加, 直到输入非数字 (例如 q) 结束
- 语义错误: 无
- 演示效果:

```
1 // boolean.c -- using a _Bool variable
2 #include <stdio.h>
3 int main(void)
4 {
5     long num;
6     long sum = 0L;
7     _Bool input_is_good;
8
9     printf("Please enter an integer to be summed ");
10    printf("(q to quit): ");
11    input_is_good = (scanf("%ld", &num) == 1);
12    while (input_is_good)
13    {
14        sum = sum + num;
15        printf("Please enter next integer (q to quit): ");
16        input_is_good = (scanf("%ld", &num) == 1);
17    }
18    printf("Those integers sum to %ld.\n", sum);
19
20    return 0;
21 }
22 ```bash
23 Hope you enjoyed this power trip -- bye!
```

sweetie1.c

- 作用: 演示 `while` 循环的基本使用方法
- 功能: 使用 `while` 循环重复输出固定文本 22 次

- 语义错误: 无
- 演示效果:

```
1 // sweetie1.c -- a counting loop
2 #include <stdio.h>
3 int main(void)
4 {
5     const int NUMBER = 22;
6     int count = 1;           // initialization
7
8     while (count ≤ NUMBER)   // test
9     {
10         printf("Be my Valentine!\n"); // action
11         count++;              // update count
12     }
13
14     return 0;
15 }
```

```
1 Be my Valentine!
2 Be my Valentine!
3 Be my Valentine!
4 Be my Valentine!
5 Be my Valentine!
6 Be my Valentine!
7 Be my Valentine!
8 Be my Valentine!
9 Be my Valentine!
10 Be my Valentine!
11 Be my Valentine!
12 Be my Valentine!
13 Be my Valentine!
14 Be my Valentine!
15 Be my Valentine!
16 Be my Valentine!
17 Be my Valentine!
18 Be my Valentine!
19 Be my Valentine!
20 Be my Valentine!
21 Be my Valentine!
```

sweetie2.c

- 作用: 使用 `for` 循环重写 sweetie1.c 中的计数逻辑
- 功能: 利用 `for` 循环结构, 在循环内打印同样的 Valentine 提示语, 共 22 次
- 语义错误: 无
- 演示效果:

```
1 // sweetie2.c -- a counting loop using for
2 #include <stdio.h>
3 int main(void)
4 {
5     const int NUMBER = 22;
6     int count;
7
8     for (count = 1; count ≤ NUMBER; count++)
9         printf("Be my Valentine!\n");
10
11     return 0;
12 }
```

```
1 Be my Valentine!
2 Be my Valentine!
3 Be my Valentine!
4 Be my Valentine!
5 Be my Valentine!
6 Be my Valentine!
7 Be my Valentine!
8 Be my Valentine!
9 Be my Valentine!
10 Be my Valentine!
11 Be my Valentine!
12 Be my Valentine!
13 Be my Valentine!
14 Be my Valentine!
15 Be my Valentine!
16 Be my Valentine!
17 Be my Valentine
18 - 作用: 演示 `while` 循环的基本使用方法
19 - 功能: 使用 `while` 循环重复输出固定文本 22 次
20 - 语义错误: 无
21 - 演示效果:
```

```

22  `` `c
23  // sweetie1.c -- a counting loop
24  #include <stdio.h>
25  int main(void)
26  {
27      const int NUMBER = 22;
28      int count = 1;                // initialization
29
30      while (count ≤ NUMBER)        // test
31      {
32          printf("Be my Valentine!\n"); // action
33          count++;                    // update count
34      }
35
36      return 0;
37  }

```

```

1  Be my Valentine!
2  Be my Valentine!
3  Be my Valentine!
4  Be my Valentine!
5  Be my Valentine!
6  Be my Valentine!
7  Be my Valentine!
8  Be my Valentine!
9  Be my Valentine!
10 Be my Valentine!
11 Be my Valentine!
12 Be my Valentine!
13 Be my Valentine!
14 Be my Valentine!
15 Be my Valentine!
16 Be my Valentine!
17 Be my Valentine
18
19 ## sweetie2.c
20 - 作用：使用 `for` 循环重写 sweetie1.c 中的计数逻辑
21 - 功能：利用 `for` 循环结构，在循环内打印同样的 Valentine 提示语，共 22 次
22 - 语义错误：无
23 - 演示效果：
24  `` `c
25  // sweetie2.c -- a counting loop using for
26  #include <stdio.h>

```

```

27 int main(void)
28 {
29     const int NUMBER = 22;
30     int count;
31
32     for (count = 1; count ≤ NUMBER; count++)
33         printf("Be my Valentine!\n");
34
35     return 0;
36 }

```

```

1 Be my Valentine!
2 Be my Valentine!
3 Be my Valentine!
4 Be my Valentine!
5 Be my Valentine!
6 Be my Valentine!
7 Be my Valentine!
8 Be my Valentine!
9 Be my Valentine!
10 Be my Valentine!
11 Be my Valentine!
12 Be my Valentine!
13 Be my Valentine!
14 Be my Valentine!
15 Be my Valentine!
16 Be my Valentine!
17 Be my Valentine!
18 Be my Valentine!
19 Be my Valentine!
20 Be my Valentine!
21 Be my Valentine!
22 Be my Valentine!

```

cmpflt.c

- 作用: 演示浮点数比较的正确方法, 通过绝对值差判断是否接近目标常数
- 功能: 循环提示用户输入 pi 的值, 直到精度范围内接近 3.14159 为止
- 语义错误: 无
- 演示效果:

```

1 // cmpflt.c -- floating-point comparisons
2 #include <math.h>
3 #include <stdio.h>
4 int main(void)
5 {
6     const double ANSWER = 3.14159;
7     double response;
8
9     printf("What is the value of pi?\n");
10    scanf("%lf", &response);
11    while (fabs(response - ANSWER) > 0.0001)
12    {
13        printf("Try again!\n");
14        scanf("%lf", &response);
15    }
16    printf("Close enough!\n");
17
18    return 0;
19 }

```

```

1 What is the value of pi?
2 Try again!
3 Close enough!

```

do_while.c

- 作用: 演示 `do while` 循环的使用方式——至少会执行一次循环体
- 功能: 循环提示用户输入秘密代码，直到输入与预设值匹配
- 语义错误: 无
- 演示效果:

```

1 /* do_while.c -- exit condition loop */
2 #include <stdio.h>
3 int main(void)
4 {
5     const int secret_code = 13;
6     int code_entered;
7
8     do
9     {

```

```

10     printf("To enter the triskaidekaphobia therapy club,\n");
11     printf("please enter the secret code number: ");
12     scanf("%d", &code_entered);
13 } while (code_entered != secret_code);
14 printf("Congratulations! You are cured!\n");
15
16 return 0;
17 }

```

```

1  To enter the triskaidekaphobia therapy club,
2  please enter the secret code number: To enter the triskaidekaphobia therapy
   club,
3  please enter the secret code number: Congratulations! You are cured!

```

rows1.c

- 作用: 演示嵌套 `for` 循环的用法
- 功能: 外层循环控制行数, 内层循环控制每行输出的字符 (从 'A' 到 'J'), 共计 6 行
- 语义错误: 无
- 演示效果:

```

1  /* rows1.c -- uses nested loops */
2  #include <stdio.h>
3  #define ROWS 6
4  #define CHARS 10
5  int main(void)
6  {
7      int row;
8      char ch;
9
10     for (row = 0; row < ROWS; row++)          /* line 10 */
11     {
12         for (ch = 'A'; ch < ('A' + CHARS); ch++) /* line 12 */
13             printf("%c", ch);
14         printf("\n");
15     }
16
17     return 0;
18 }

```

```
1 | ABCDEFGHIJ
2 | ABCDEFGHIJ
3 | ABCDEFGHIJ
4 | ABCDEFGHIJ
5 | ABCDEFGHIJ
6 | ABCDEFGHIJ
```

for_13s.c

- 作用: 演示 `for` 循环的使用——以步长 13 从 2 开始递增并输出
- 功能: 使用 `for` 循环输出一系列按 13 递增的整数值, 直到 `n` 达到 60 停止
- 语义错误: 无
- 演示效果:

```
1 | /* for_13s.c */
2 | #include <stdio.h>
3 | int main(void)
4 | {
5 |     int n;          // count by 13s from 2
6 |
7 |     for (n = 2; n < 60; n = n + 13)
8 |         printf("%d \n", n);
9 |     return 0;
10 | }
```

```
1 | 2
2 | 15
3 | 28
4 | 41
5 | 54
```

for_char.c

- 作用: 演示 `for` 循环结合字符类型变量的遍历方式
- 功能: 从字符 'a' 输出到 'z', 并打印每个字符对应的 ASCII 值
- 语义错误: 无
- 演示效果:


```

1  /* for_char.c */
2  #include <stdio.h>
3  int main(void)
4  {
5      char ch;
6
7      for (ch = 'a'; ch ≤ 'z'; ch++)
8          printf("The ASCII value for %c is %d.\n", ch, ch);
9      return 0;
10 }

```

```

1  The ASCII value for a is 97.
2  The ASCII value for b is 98.
3  The ASCII value for c is 99.
4  The ASCII value for d is 100.
5  The ASCII value for e is 101.
6  The ASCII value for f is 102.
7  The ASCII value for g is 103.
8  The ASCII value for h is 104.
9  The ASCII value for i is 105.
10 The ASCII value for j is 106.
11 The ASCII value for k is 107.
12 The ASCII value for l is 108.
13 The ASCII value for m is 109.
14 The ASCII value for n is 110.
15 The ASCII value for o is 111.
16 The ASCII value for p is 112.
17 The ASCII value for q is 113.
18 The ASCII value for r is 114.
19 The ASCII value for s is 115.
20 The ASCII value for t is 116.
21 The ASCII value for u is 117.
22 The ASCII value for v is 118.
23 The ASCII value for w is 119.
24 The ASCII value for x is 120.
25 The ASCII value for y is 121.
26 The ASCII value for z is 122.

```

for_cube.c

- 作用: 演示 `for` 循环的数值计算与表格输出

- 功能: 输出 1 到 6 的整数及其立方值, 示例了循环中的计算逻辑与格式化输出
- 语义错误: 无
- 演示效果:

```
1  /* for_cube.c -- using a for loop to make a table of cubes */
2  #include <stdio.h>
3  int main(void)
4  {
5      int num;
6
7      printf("    n    n cubed\n");
8      for (num = 1; num ≤ 6; num++)
9          printf("%5d %5d\n", num, num*num*num);
10
11     return 0;
12 }
```

1	n	n cubed
2	1	1
3	2	8
4	3	27
5	4	64
6	5	125
7	6	216

for_down.c

- 作用: 演示 `for` 循环的倒序计数方式
- 功能: 从 5 倒数到 1, 每次打印剩余秒数, 最后输出“点火”提示
- 语义错误: 无
- 演示效果:

```

1  /* for_down.c */
2  #include <stdio.h>
3  int main(void)
4  {
5      int secs;
6
7      for (secs = 5; secs > 0; secs--)
8          printf("%d seconds!\n", secs);
9      printf("We have ignition!\n");
10     return 0;
11 }

```

```

1  5 seconds!
2  4 seconds!
3  3 seconds!
4  2 seconds!
5  1 seconds!
6  We have ignition!

```

for_geo.c

- 作用: 演示 `for` 循环的几何级数递增方式
- 功能: 模拟债务每轮利息增加 10%，并打印每次的债务总额，直到债务达到或超过 150
- 语义错误: 无
- 演示效果:

```

1  /* for_geo.c */
2  #include <stdio.h>
3  int main(void)
4  {
5      double debt;
6
7      for (debt = 100.0; debt < 150.0; debt = debt * 1.1)
8          printf("Your debt is now $%.2f.\n", debt);
9      return 0;
10 }

```

```
1 Your debt is now $100.00.
2 Your debt is now $110.00.
3 Your debt is now $121.00.
4 Your debt is now $133.10.
5 Your debt is now $146.41.
```

for_none.c

- 作用: 演示 `for` 循环中省略初始化和更新语句的用法
- 功能: 用 `for` 循环没有显式的初始化和步进更新部分, 仅保留循环条件, 计算 `ans` 的值直到大于 25
- 语义错误: 无
- 演示效果:

```
1  /* for_none.c */
2  #include <stdio.h>
3  int main(void)
4  {
5      int ans, n;
6
7      ans = 2;
8      for (n = 3; ans ≤ 25; )
9          ans = ans * n;
10     printf("n = %d; ans = %d.\n", n, ans);
11     return 0;
12 }
```

```
1  n = 3; ans = 54.
```

summing.c

- 作用: 演示 `while` 循环对交互输入数据的加总运算
- 功能: 从用户交互输入中连续读取整数进行累加, 直到输入非数字 (例如 `q`) 结束, 最后输出总和
- 语义错误: 无
- 演示效果:

```
1  /* summing.c -- sums integers entered interactively */
2  #include <stdio.h>
```

```

3  int main(void)
4  {
5      long num;
6      long sum = 0L;      /* initialize sum to zero */
7      int status;
8
9      printf("Please enter an integer to be summed ");
10     printf("(q to quit): ");
11     status = scanf("%ld", &num);
12     while (status == 1) /* == means "is equal to" */
13     {
14         sum = sum + num;
15         printf("Please enter next integer (q to quit): ");
16         status = scanf("%ld", &num);
17     }
18     printf("Those integers sum to %ld.\n", sum);
19
20     return 0;
21 }

```

```

1  Please enter an integer to be summed (q to quit): Please enter next integer (q
   to quit): Please enter next integer (q to quit): Please enter next integer (q
   to quit): Please enter next integer (q to quit): Please enter next integer (q
   to quit): Those integers sum to 15.

```

for_show.c

- 作用: 演示 `for` 循环中初始化部分可用于执行一次性语句
- 功能: 在 `for` 语句的初始化部分直接调用 `printf()` 输出提示信息, 通过 `scanf` 持续输入数字直到符合条件结束
- 语义错误: 无
- 演示效果:

```

1  /* for_show.c */
2  #include <stdio.h>
3  int main(void)
4  {
5      int num = 0;
6
7      for (printf("Keep entering numbers!\n"); num != 6; )
8          scanf("%d", &num);
9      printf("That's the one I want!\n");
10     return 0;
11 }

```

```

1  Keep entering numbers!
2  That's the one I want!

```

scores_in.c

- 作用: 演示循环处理数组数据的过程
- 功能: 从用户输入读取 10 个高尔夫成绩, 输出成绩列表, 计算总分与平均分, 并根据平均分计算 handicap (与常量 PAR 的差值)
- 语义错误: 无
- 演示效果:

```

1  // scores_in.c -- uses loops for array processing
2  #include <stdio.h>
3  #define SIZE 10
4  #define PAR 72
5  int main(void)
6  {
7      int index, score[SIZE];
8      int sum = 0;
9      float average;
10
11     printf("Enter %d golf scores:\n", SIZE);
12     for (index = 0; index < SIZE; index++)
13         scanf("%d", &score[index]); // read in the ten scores
14     printf("The scores read in are as follows:\n");
15     for (index = 0; index < SIZE; index++)
16         printf("%5d", score[index]); // verify input

```

```

17     printf("\n");
18     for (index = 0; index < SIZE; index++)
19         sum += score[index];           // add them up
20     average = (float) sum / SIZE;      // time-honored method
21     printf("Sum of scores = %d, average = %.2f\n", sum, average);
22     printf("That's a handicap of %.0f.\n", average - PAR);
23
24     return 0;
25 }

```

```

1  Enter 10 golf scores:
2  The scores read in are as follows:
3      70   71   72   73   74   75   76   77   78   79
4  Sum of scores = 745, average = 74.50
5  That's a handicap of 2.

```

for_wild.c

- 作用: 演示 `for` 循环中步进表达式的复杂变化
- 功能: 初始化 `y=55`, 循环中 `x` 从 1 开始递增, `y` 根据公式 `(++x * 5) + 50` 更新, 直到 `y` 超过 75
- 语义错误: 无
- 演示效果:

```

1  /* for_wild.c */
2  #include <stdio.h>
3  int main(void)
4  {
5      int x;
6      int y = 55;
7
8      for (x = 1; y ≤ 75; y = (++x * 5) + 50)
9          printf("%10d %10d\n", x, y);
10     return 0;
11 }

```

1	1	55
2	2	60
3	3	65
4	4	70
5	5	75

rows2.c

- 作用: 演示嵌套 `for` 循环中内层循环的起始值依赖于外层循环变量
- 功能: 外层循环控制行数, 内层循环从 `'A' + row` 开始输出字符到 `'A' + CHARS`, 形成递减长度的字符序列
- 语义错误: 无
- 演示效果:

```
1 // rows2.c -- using dependent nested loops
2 #include <stdio.h>
3 int main(void)
4 {
5     const int ROWS = 6;
6     const int CHARS = 6;
7     int row;
8     char ch;
9
10    for (row = 0; row < ROWS; row++)
11    {
12        for (ch = ('A' + row); ch < ('A' + CHARS); ch++)
13            printf("%c", ch);
14        printf("\n");
15    }
16
17    return 0;
18 }
```

1	ABCDEF
2	BCDEF
3	CDEF
4	DEF
5	EF
6	F

postage.c

- 作用: 演示在 `for` 循环中计算并打印邮资价格表
- 功能: 从1盎司开始, 使用常量 `FIRST_OZ` 和 `NEXT_OZ` 计算每增加一盎司的邮资, 直到16盎司
- 语义错误: 无
- 演示效果:

```
1 // postage.c -- first-class postage rates
2 #include <stdio.h>
3 int main(void)
4 {
5     const int FIRST_OZ = 46; // 2013 rate
6     const int NEXT_OZ = 20;  // 2013 rate
7     int ounces, cost;
8
9     printf(" ounces  cost\n");
10    for (ounces=1, cost=FIRST_OZ; ounces ≤ 16; ounces++,
11         cost += NEXT_OZ)
12        printf("%5d   $%4.2f\n", ounces, cost/100.0);
13
14    return 0;
15 }
```

1	ounces	cost
2	1	\$0.46
3	2	\$0.66
4	3	\$0.86
5	4	\$1.06
6	5	\$1.26
7	6	\$1.46
8	7	\$1.66
9	8	\$1.86
10	9	\$2.06
11	10	\$2.26
12	11	\$2.46
13	12	\$2.66
14	13	\$2.86
15	14	\$3.06
16	15	\$3.26
17	16	\$3.46

power.c

- 作用: 演示函数调用与数学幂运算
- 功能: 读取一个浮点数与一个正整数作为指数, 调用 power 函数计算 n 的 p 次幂并输出
- 语义错误: 无
- 演示效果:

```
1 // power.c -- raises numbers to integer powers
2 #include <stdio.h>
3 double power(double n, int p); // ANSI prototype
4 int main(void)
5 {
6     double x, xpow;
7     int exp;
8
9     printf("Enter a number and the positive integer power");
10    printf(" to which\nthe number will be raised. Enter q");
11    printf(" to quit.\n");
12    while (scanf("%lf%d", &x, &exp) == 2)
13    {
14        xpow = power(x, exp); // function call
15        printf("%.3g to the power %d is %.5g\n", x, exp, xpow);
16        printf("Enter next pair of numbers or q to quit.\n");
17    }
18    printf("Hope you enjoyed this power trip -- bye!\n");
19
20    return 0;
21 }
22
23 double power(double n, int p) // function definition
24 {
25     double pow = 1;
26     int i;
27
28     for (i = 1; i ≤ p; i++)
29         pow *= n;
30
31     return pow; // return the value of pow
32 }
```

```
1 Enter a number and the positive integer power to which
2 the number will be raised. Enter q to quit.
3 2 to the power 3 is 8
4 Enter next pair of numbers or q to quit.
5 Hope you enjoyed this power trip -- bye!
```

entry.c

- 作用: 演示 `while` 循环的入口条件用法, 在满足条件前反复执行循环体
- 功能: 循环提示用户输入秘密代码, 若错误则继续提示直到输入正确
- 语义错误: 无
- 演示效果:

```
1  /* entry.c -- entry condition loop */
2  #include <stdio.h>
3  int main(void)
4  {
5      const int secret_code = 13;
6      int code_entered;
7
8      printf("To enter the triskaidekaphobia therapy club,\n");
9      printf("please enter the secret code number: ");
10     scanf("%d", &code_entered);
11     while (code_entered != secret_code)
12     {
13         printf("To enter the triskaidekaphobia therapy club,\n");
14         printf("please enter the secret code number: ");
15         scanf("%d", &code_entered);
16     }
17     printf("Congratulations! You are cured!\n");
18
19     return 0;
20 }
```

```
1 To enter the triskaidekaphobia therapy club,
2 please enter the secret code number: To enter the triskaidekaphobia therapy
  club,
3 please enter the secret code number: Congratulations! You are cured!
```

t_and_f.c

- 作用: 演示 C 语言中关系表达式的布尔值表示
- 功能: 展示一个为真的表达式和一个为假的表达式在 C 中的数值表示 (true 为 1, false 为 0)
- 语义错误: 无
- 演示效果:

```
1  /* t_and_f.c -- true and false values in C */
2  #include <stdio.h>
3  int main(void)
4  {
5      int true_val, false_val;
6
7      true_val = (10 > 2);    // value of a true relationship
8      false_val = (10 == 2); // value of a false relationship
9      printf("true = %d; false = %d \n", true_val, false_val);
10
11     return 0;
12 }
```

```
1  true = 1; false = 0
```

trouble.c

- 作用: 演示赋值运算符与比较运算符的区别, 修复错误避免无限循环
- 功能: 循环读取用户输入的整数并累加, 直到输入非数字 (例如 q) 结束
- 主要错误: 使用 `=` 替代 `==` 会导致循环条件永远为真而无限循环
- 演示效果:

```
1  // trouble.c -- misuse of =
2  // will cause infinite loop
3  #include <stdio.h>
4  int main(void)
5  {
6      long num;
7      long sum = 0L;
8      int status;
9  }
```

```

10     printf("Please enter an integer to be summed ");
11     printf("(q to quit): ");
12     status = scanf("%ld", &num);
13     while (status == 1)
14     {
15         sum = sum + num;
16         printf("Please enter next integer (q to quit): ");
17         status = scanf("%ld", &num);
18     }
19     printf("Those integers sum to %ld.\n", sum);
20
21     return 0;
22 }

```

```

1 | Please enter an integer to be summed (q to quit): Please enter next integer (q
  | to quit): Please enter next integer (q to quit): Please enter next integer (q
  | to quit): Those integers sum to 6.

```

truth.c

- 作用: 演示 C 语言中整数在布尔条件中的求值方式, 非零为真, 零为假
- 功能: 使用 while 循环测试不同整数值的真值性, 展示正数、负数在条件判断中的效果
- 语义错误: 无
- 演示效果:

```

1 | // truth.c -- what values are true?
2 | #include <stdio.h>
3 | int main(void)
4 | {
5 |     int n = 3;
6 |
7 |     while (n)
8 |         printf("%2d is true\n", n--);
9 |     printf("%2d is false\n", n);
10 |
11 |     n = -3;
12 |     while (n)
13 |         printf("%2d is true\n", n++);
14 |     printf("%2d is false\n", n);
15 |

```

```
16     return 0;
17 }
```

```
1 3 is true
2 2 is true
3 1 is true
4 0 is false
5 -3 is true
6 -2 is true
7 -1 is true
8 0 is false
```

when.c

- 作用: 演示 `while` 循环的退出条件, 以及循环变量在循环中的变化过程
- 功能: 从 $n=5$ 开始, 当 n 小于 7 时进入循环, 每次循环先输出当前值, 再递增 n 并输出更新后的值, 直到条件为假退出
- 语义错误: 无
- 演示效果:

```
1 // when.c -- when a loop quits
2 #include <stdio.h>
3 int main(void)
4 {
5     int n = 5;
6
7     while (n < 7)                // line 7
8     {
9         printf("n = %d\n", n);
10        n++;                      // line 10
11        printf("Now n = %d\n", n); // line 11
12    }
13    printf("The loop has finished.\n");
14
15    return 0;
16 }
```

```
1 | n = 5
2 | Now n = 6
3 | n = 6
4 | Now n = 7
5 | The loop has finished.
```

while1.c

- 作用: 演示 `while` 循环中使用花括号以防止无限循环
- 功能: 从 $n=0$ 开始, 当 n 小于 3 时进入循环, 打印当前值并递增, 循环结束后打印完成信息
- 主要错误: 原代码中缺少花括号, 导致递增语句在循环外执行, 形成死循环
- 演示效果:

```
1 | /* while1.c -- watch your braces          */
2 | /* bad coding creates an infinite loop */
3 | #include <stdio.h>
4 | int main(void)
5 | {
6 |     int n = 0;
7 |
8 |     while (n < 3)
9 |     {
10 |         printf("n is %d\n", n);
11 |         n++;
12 |     }
13 |     printf("That's all this program does\n");
14 |
15 |     return 0;
16 | }
```

```
1 | n is 0
2 | n is 1
3 | n is 2
4 | That's all this program does
```

while2.c

- 作用: 演示 `while` 循环语句中分号的影响

- 功能: 使用 `while (n++ < 3);` 语句实际运行了空循环体, 循环结束后再执行 `printf` 打印 `n` 的值
- 语义错误: 分号导致循环体为空, `n` 在循环中递增至 4, 循环结束后才打印结果
- 演示效果:

```

1  /* while2.c -- watch your semicolons */
2  #include <stdio.h>
3  int main(void)
4  {
5      int n = 0;
6
7      while (n++ < 3);          /* line 7 */
8      printf("n is %d\n", n); /* line 8 */
9      printf("That's all this program does.\n");
10
11     return 0;
12 }

```

```

1  n is 4
2  That's all this program does.

```

zeno.c

- 作用: 演示循环累加计算序列级数 (Zeno's paradox) 的逼近行为
- 功能: 对输入的项数执行循环, 每次将 1 除以 2 的幂次累加, 展示时间趋近极限的过程
- 语义错误: 无
- 演示效果:

```

1  /* zeno.c -- series sum */
2  #include <stdio.h>
3
4  int main(void)
5  {
6      int t_ct;          // term count
7      double time, power_of_2;
8      int limit;
9
10     printf("Enter the number of terms you want: ");
11     scanf("%d", &limit);
12     for (time=0, power_of_2=1, t_ct=1; t_ct ≤ limit;

```



```

13         t_ct++, power_of_2 *= 2.0)
14     {
15         time += 1.0/power_of_2;
16         printf("time = %f when terms = %d.\n", time, t_ct);
17     }
18
19     return 0;
20 }

```

```

1 Enter the number of terms you want: time = 1.000000 when terms = 1.
2 time = 1.500000 when terms = 2.
3 time = 1.750000 when terms = 3.
4 time = 1.875000 when terms = 4.
5 time = 1.937500 when terms = 5.

```

- 作用: 演示 `while` 循环语句中分号的影响
- 功能: 使用 `while (n++ < 3);` 语句实际运行了空循环体, 循环结束后再执行 `printf` 打印 `n` 的值
- 语义错误: 分号导致循环体为空, `n` 在循环中递增至 4, 循环结束后才打印结果
- 演示效果:

```

1  /* while2.c -- watch your semicolons */
2  #include <stdio.h>
3  int main(void)
4  {
5      int n = 0;
6
7      while (n++ < 3);          /* line 7 */
8      printf("n is %d\n", n); /* line 8 */
9      printf("That's all this program does.\n");
10
11     return 0;
12 }

```

```

1 n is 4
2 That's all this program does.

```

- 作用: 演示 `while` 循环中使用花括号以防止无限循环
- 功能: 从 `n=0` 开始, 当 `n` 小于 3 时进入循环, 打印当前值并递增, 循环结束后打印完成信息

- 主要错误: 原代码中缺少花括号, 导致递增语句在循环外执行, 形成死循环
- 演示效果:

```
1  /* while1.c -- watch your braces      */
2  /* bad coding creates an infinite loop */
3  #include <stdio.h>
4  int main(void)
5  {
6      int n = 0;
7
8      while (n < 3)
9      {
10         printf("n is %d\n", n);
11         n++;
12     }
13     printf("That's all this program does\n");
14
15     return 0;
16 }
```

```
1  n is 0
2  n is 1
3  n is 2
4  That's all this program does
```

- 作用: 演示 `while` 循环的退出条件, 以及循环变量在循环中的变化过程
- 功能: 从 $n=5$ 开始, 当 n 小于 7 时进入循环, 每次循环先输出当前值, 再递增 n 并输出更新后的值, 直到条件为假退出
- 语义错误: 无
- 演示效果:

```
1  // when.c -- when a loop quits
2  #include <stdio.h>
3  int main(void)
4  {
5      int n = 5;
6
7      while (n < 7)                // line 7
8      {
9          printf("n = %d\n", n);
```

```

10         n++; // line 10
11         printf("Now n = %d\n", n); // line 11
12     }
13     printf("The loop has finished.\n");
14
15     return 0;
16 }

```

```

1  n = 5
2  Now n = 6
3  n = 6
4  Now n = 7
5  The loop has finished.

```

- 作用: 演示 C 语言中整数在布尔条件中的求值方式，非零为真，零为假
- 功能: 使用 while 循环测试不同整数值的真值性，展示正数、负数在条件判断中的效果
- 语义错误: 无
- 演示效果:

```

1  // truth.c -- what values are true?
2  #include <stdio.h>
3  int main(void)
4  {
5      int n = 3;
6
7      while (n)
8          printf("%2d is true\n", n--);
9      printf("%2d is false\n", n);
10
11     n = -3;
12     while (n)
13         printf("%2d is true\n", n++);
14     printf("%2d is false\n", n);
15
16     return 0;
17 }

```

```
1 3 is true
2 2 is true
3 1 is true
4 0 is false
5 -3 is true
6 -2 is true
7 -1 is true
8 0 is false
```

- 作用: 演示赋值运算符与比较运算符的区别, 修复错误避免无限循环
- 功能: 循环读取用户输入的整数并累加, 直到输入非数字 (例如 q) 结束
- 主要错误: 使用 `=` 替代 `==` 会导致循环条件永远为真而无限循环
- 演示效果:

```
1 // trouble.c -- misuse of =
2 // will cause infinite loop
3 #include <stdio.h>
4 int main(void)
5 {
6     long num;
7     long sum = 0L;
8     int status;
9
10    printf("Please enter an integer to be summed ");
11    printf("(q to quit): ");
12    status = scanf("%ld", &num);
13    while (status == 1)
14    {
15        sum = sum + num;
16        printf("Please enter next integer (q to quit): ");
17        status = scanf("%ld", &num);
18    }
19    printf("Those integers sum to %ld.\n", sum);
20
21    return 0;
22 }
```

```
1 | Please enter an integer to be summed (q to quit): Please enter next integer (q
   | to quit): Please enter next integer (q to quit): Please enter next integer (q
   | to quit): Those integers sum to 6.
```

- 作用: 演示 C 语言中关系表达式的布尔值表示
- 功能: 展示一个为真的表达式和一个为假的表达式在 C 中的数值表示 (true 为 1, false 为 0)
- 语义错误: 无
- 演示效果:

```
1 | /* t_and_f.c -- true and false values in C */
2 | #include <stdio.h>
3 | int main(void)
4 | {
5 |     int true_val, false_val;
6 |
7 |     true_val = (10 > 2);    // value of a true relationship
8 |     false_val = (10 == 2); // value of a false relationship
9 |     printf("true = %d; false = %d \n", true_val, false_val);
10 |
11 |     return 0;
12 | }
```

```
1 | true = 1; false = 0
```

- 作用: 演示 `while` 循环的入口条件用法, 在满足条件前反复执行循环体
- 功能: 循环提示用户输入秘密代码, 若错误则继续提示直到输入正确
- 语义错误: 无
- 演示效果:

```
1 | /* entry.c -- entry condition loop */
2 | #include <stdio.h>
3 | int main(void)
4 | {
5 |     const int secret_code = 13;
6 |     int code_entered;
7 |
8 |     printf("To enter the triskaidekaphobia therapy club,\n");
9 |     printf("please enter the secret code number: ");
```

```
10     scanf("%d", &code_entered);
11     while (code_entered != secret_code)
12     {
13         printf("To enter the triskaidekaphobia therapy club,\n");
14         printf("please enter the secret code number: ");
15         scanf("%d", &code_entered);
16     }
17     printf("Congratulations! You are cured!\n");
18
19     return 0;
20 }
```

```
1  To enter the triskaidekaphobia therapy club,
2  please enter the secret code number: To enter the triskaidekaphobia therapy
   club,
3  please enter the secret code number: Congratulations! You are cured!
```